

02689 Advanced Numerical Methods for Differential Equations

Assignment #1: Spectral Approximations and Operations

Viggo Starcke
s250690@dtu.dk

Fourier Spectral Methods

a) To find the exact Fourier coefficients of $u(x)$ we use the continuous Fourier coefficient formula

$$\hat{u}_k = \frac{1}{T} \int_0^T u(x) e^{-ik \frac{2\pi}{T} x} dx$$

where T is the length of our domain. For our domain $x \in [0, 2]$ we transform to that of a full circle $\theta \in [0, 2\pi]$ such that $\theta = \pi x$. The derivative $d\theta = \pi dx$ means we can write $dx = \frac{d\theta}{\pi}$. We get

$$\hat{u}_k = \frac{1}{2\pi} \int_0^{2\pi} \frac{e^{-ik\theta}}{2 - \cos(\theta)} d\theta$$

To map θ into a unit circle we set $z = e^{i\theta}$, and use Euler's formula $e^{i\theta} = \cos(\theta) + i \sin(\theta)$ (or for negative angle $e^{-i\theta} = \cos(\theta) - i \sin(\theta)$) to find an expression for $\cos(\theta)$;

$$\begin{aligned} e^{i\theta} + e^{-i\theta} &= 2 \cos(\theta) \\ \cos(\theta) &= \frac{e^{i\theta} + e^{-i\theta}}{2} \\ \cos(\theta) &= \frac{z + z^{-1}}{2} \end{aligned}$$

We also need to express $d\theta$ in terms of z . The derivative of z wrt θ is $\frac{dz}{d\theta} = ie^{i\theta}$ which rearranges to the term for $d\theta = \frac{dz}{ie^{i\theta}} = \frac{dz}{iz}$. Substituting this back into our integral we get

$$\hat{u}_k = \frac{1}{2\pi} \int_0^{2\pi} \frac{z^{-k}}{2 - \frac{z+z^{-1}}{2}} \frac{dz}{iz}$$

For this, we can simplify the denominator;

$$2 - \frac{z + z^{-1}}{2} = \frac{4}{2} - \frac{z + z^{-1}}{2} = \frac{4 - z - z^{-1}}{2} = \frac{4 - z - \frac{1}{z}}{2} = \frac{4z - z^2 - 1}{2z}$$

and substitute back into the formula;

$$\hat{u}_k = \frac{1}{2\pi} \int_0^{2\pi} \frac{z^{-k}}{\frac{4z - z^2 - 1}{2z}} \frac{dz}{iz} = \frac{1}{2\pi} \int_0^{2\pi} \frac{z^{-k} \cdot 2z}{4z - z^2 - 1} \frac{dz}{iz} = \int_0^{2\pi} \frac{1 \cdot z^{-k} \cdot 2z}{2\pi \cdot (4z - z^2 - 1) \cdot iz} dz = \frac{1}{2\pi i} \int_0^{2\pi} \frac{2z^{-k}}{4z - z^2 - 1} dz$$

For our new domain, θ goes from 0 to 2π as our substitution $z = e^{i\theta}$ goes around the unit circle. So we write

$$\hat{u}_k = \frac{1}{2\pi i} \oint_{|z|=1} \frac{2z^{-k}}{4z - z^2 - 1} dz$$

To integrate a our formula within this closed loop, we refer to Cauchy's Residue Theorem, which states that the integral for a closed loop can be formulated as the sum of residues of the poles (corresponds to the points where the denominator in our function equals zero). The denominator is now a quadratic equation, so we solve this for $z = 2 \pm \sqrt{3}$. Note that only $z_2 = 2 - \sqrt{3}$ is inside our closed loop, as it is below 1. The residue of a pole for a function $f(z) = \frac{p(z)}{q(z)}$ is calculated as $\frac{p(z)}{q'(z)}$. The derivative of our denominator; $q'(z) = 4 - 2z$. We then evaluate the value of our integral for $z = 2 - \sqrt{3}$ and multiply by $2\pi i$ as Cauchy's Residue Theorem states;

$$\hat{u}_k = 2\pi i \left(\frac{1}{2\pi i} \cdot \frac{2(2 - \sqrt{3})^{-k}}{4 - 2(2 - \sqrt{3})} \right) = \frac{2(2 - \sqrt{3})^{-k}}{2\sqrt{3}} = \frac{(2 - \sqrt{3})^{-k}}{\sqrt{3}}$$

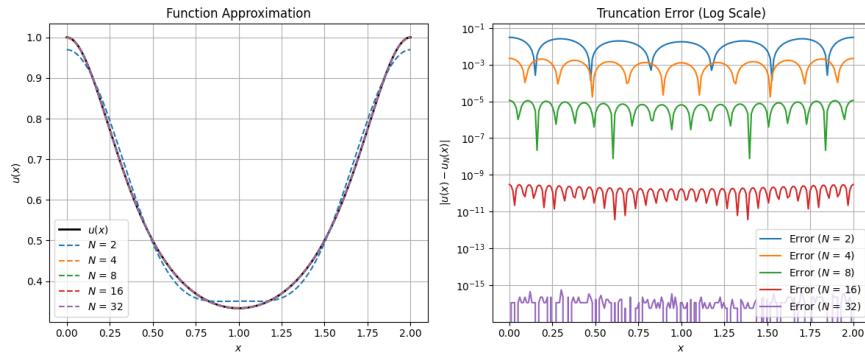
To match the given result from the blackboard, we can rewrite $(2 - \sqrt{3})^{-k}$ as $(2 + \sqrt{3})^k$, and divide by this term. Further, we can argue that $u(x)$ is a symmetric function such that $\hat{u}_k$ is also symmetric. Thus, it is more precise to force k into an absolute value. We then have;

$$\hat{u}_k = \frac{1}{\sqrt{3}(2 + \sqrt{3})^{|k|}}$$

As k grows in this formula, the size of the Fourier coefficient shrinks exponentially at rate of decay by $(2 + \sqrt{3})^{|k|}$. If we reconstruct the function using a truncated Fourier series

$$u_N(x) = \sum_{k=-N}^N \hat{u}_k e^{ik\pi x}$$

we can measure the error rate as the truncation error $E_N(x) = |u(x) - u_N(x)|$. But as we know the rate of decay is exponential for the coefficients $\hat{u}_k$, we also know that the truncation error will shrink equivalently exponentially as N grows. We confirm this in plotting our numerical Fourier series;



b) We recall the formula for discrete Fourier transform;

$$\tilde{u}_k = \frac{1}{N} \sum_{j=0}^{N-1} u(x_j) e^{-ik\pi x_j}$$

When we evaluate this continuous function on a discrete grid, we can not distinguish low frequencies from high frequencies. It is called aliasing and essentially we say that $\tilde{u}_k$ is not only equal to $\hat{u}_k$ but also the sum of every other higher frequency alias, $\hat{u}_{k+mN}$ for $m \in \mathbb{Z}$. This mechanism means that the lower N is, the higher the noise from the aliases is. For $N = 4$ we see that the "noisiest" alias is consequently $\hat{u}_{2+1.4}$, which is way larger coefficient than for $N = 16$, i.e. $\hat{u}_{2+1.4} \gg \hat{u}_{2+1.16}$. The absolute difference between our exact value $\hat{u}_k$ and the truncated discrete approximation is thereby determined by the sum of these aliases. As we compute the discrete Fourier coefficients and measure the absolute difference we see that by $N = 32$ the noise is at a machine precision level. Since $u(x) \in C^\infty$ we say that this is spectral convergence.

N	Discrete coefficient	Error (absolute difference)
4	0.0833	$4.19 \cdot 10^{-2}$
8	0.0417	$2.15 \cdot 10^{-4}$
16	0.0415	$5.71 \cdot 10^{-9}$
32	0.0415	$4.14 \cdot 10^{-17}$
64	0.0415	$4.22 \cdot 10^{-17}$

c) We start with the standard definition of a discrete Fourier interpolant evaluated on a 2π domain, which reconstructs the function $u(x)$ from its discrete frequency coefficients $\tilde{u}_k$;

$$I_N u(x) = \sum_{k=-N/2}^{N/2} \frac{1}{\bar{c}_k} \tilde{u}_k e^{ikx}$$

where $\bar{c}_k = 2$ for the boundaries $k = \pm \frac{N}{2}$ and $\bar{c}_k = 1$ else. We substitute the definition for a discrete Fourier coefficient $\tilde{u}_k$ into the interpolant;

$$I_N u(x) = \sum_{k=-N/2}^{N/2} \frac{1}{\bar{c}_k} \left(\frac{1}{N} \sum_{j=0}^{N-1} u(x_j) e^{-ik\pi x_j} \right) e^{ikx}$$

And to express this in the form of a Lagrange interpolation, $I_N u(x) = \sum_{j=0}^{N-1} u(x_j) h_j(x)$, we swap the order of summation to isolate everything that depends on k ;

$$I_N u(x) = \sum_{j=0}^{N-1} u(x_j) \underbrace{\left[\frac{1}{N} \sum_{k=-N/2}^{N/2} \frac{1}{\bar{c}_k} e^{ik(x-x_j)} \right]}_{h_j(x)}$$

We have our Lagrange polynomial isolated as $h_j(x)$, which we simplify by defining $\theta = x - x_j$ such that we can define $h_j(\theta)$;

$$h_j(\theta) = \frac{1}{N} \sum_{k=-N/2}^{N/2} \frac{1}{\bar{c}_k} e^{ik\theta}$$

Now, because $\bar{c}_k = 2$ only at the boundaries (and $\bar{c}_k = 1$ everywhere else), we can write the boundary cases out of the summation and get rid of $\bar{c}_k$;

$$h_j(\theta) = \frac{1}{N} \left(\frac{e^{i\frac{N}{2}\theta} + e^{-i\frac{N}{2}\theta}}{2} + \sum_{k=-N/2+1}^{N/2-1} e^{ik\theta} \right)$$

From Euler's formula the first term in the braces simplifies to $\frac{e^{i\frac{N}{2}\theta} + e^{-i\frac{N}{2}\theta}}{2} = \cos\left(\frac{N}{2}\theta\right)$;

$$h_j(\theta) = \frac{1}{N} \left(\cos\left(\frac{N}{2}\theta\right) + \sum_{k=-N/2+1}^{N/2-1} e^{ik\theta} \right)$$

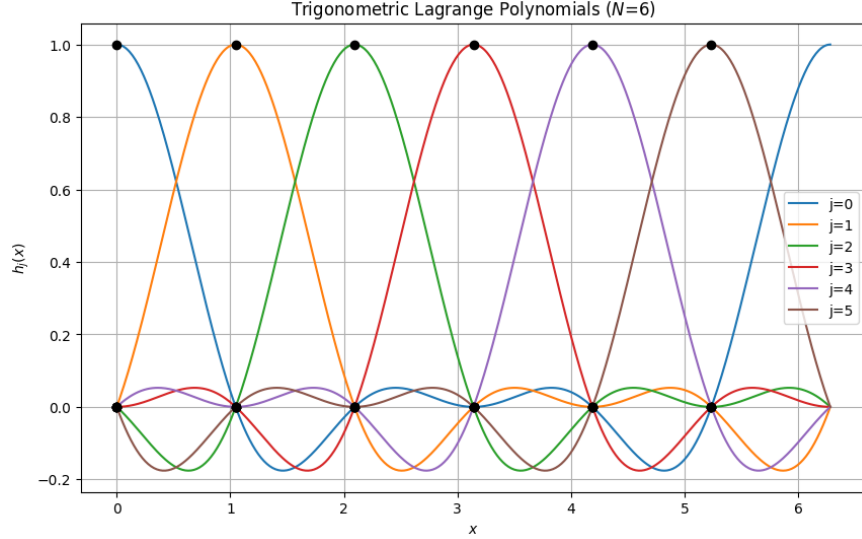
The remaining summation is now a finite geometric series, which we can write as $\sum_{k=-N/2+1}^{N/2-1} e^{ik\theta} = \frac{e^{i\theta(-N/2+1)} - e^{i\theta(N/2)}}{1 - e^{i\theta}} = \frac{\sin\left(\frac{N-1}{2}\theta\right)}{\sin\left(\frac{\theta}{2}\right)}$. So we substitute this into our expression for $h_j(\theta)$;

$$h_j(\theta) = \frac{1}{N} \left(\cos\left(\frac{N}{2}\theta\right) + \frac{\sin\left(\frac{N-1}{2}\theta\right)}{\sin\left(\frac{\theta}{2}\right)} \right)$$

Or with $x - x_j = \theta$ substituted back in;

$$h_j(x) = \frac{1}{N} \left(\cos \left(\frac{N}{2}(x - x_j) \right) + \frac{\sin \left(\frac{N-1}{2}(x - x_j) \right)}{\sin \left(\frac{(x-x_j)}{2} \right)} \right)$$

This our expression for the Lagrange polynomials, which we plot on a uniform grid as described with 6 nodal points, to verify that $h_j(x_i) = \delta_{ij}$, each function is equal to 1 at their own node and 0 at all other nodes;



The differentiation matrix $\mathcal{D}$ is given as the derivative of the Fourier interpolant of $h_j(x)$ at every grid point x_i for N grid points in total. This gives us a $N \times N$ matrix where every element $\mathcal{D}_{ij} = h'_j(x_i)$. The derivative of $h_j(x)$ is;

$$h'_j(x) = \frac{1}{N} \left[-\frac{N}{2} \sin \left(\frac{N}{2}(x - x_j) \right) + \frac{\frac{N-1}{2} \cos \left(\frac{N-1}{2}(x - x_j) \right) \sin \left(\frac{x-x_j}{2} \right) - \frac{1}{2} \sin \left(\frac{N-1}{2}(x - x_j) \right) \cos \left(\frac{x-x_j}{2} \right)}{\sin^2 \left(\frac{x-x_j}{2} \right)} \right]$$

The diagonal elements, where $i = j$, represents the derivative of $h_j(x)$ evaluated at its own node, $h'_j(x_j)$. Because $h_j(x)$ is a symmetric wave centered around x_j , the slope at this point is zero; $\mathcal{D}_{i,i} = 0$.

d) Since the domain is shifted from $\theta \in [0, 2\pi]$ back to $x \in [0, 2]$ we need to define some constant c such that $x = c\theta$. We set $c = \frac{1}{\pi}$ such that for the upper boundary we get $x = \frac{1}{\pi}(2\pi) = 2$ for $\theta = 2\pi$. This gives that $x = \frac{1}{\pi}\theta$ or $\theta = x\pi$.

Our differentiation matrix $\mathcal{D}$ applied to some vector of function values v calculates the derivatives wrt θ ; $\mathcal{D}v = \frac{dv}{d\theta}$. But as we want to compute it for the x , $\frac{dv}{dx}$, we arrange according to the chain rule such that $\frac{dv}{dx} = \frac{dv}{d\theta} \cdot \frac{d\theta}{dx}$, where $\frac{d\theta}{dx} = \pi$ (from $\theta = \pi x$). Thereby we have that $\frac{dv}{dx} = \mathcal{D}v \cdot \pi$. We compute this for $v(x)$ on $N = 4, 8, 16, \dots, 64$, and measure the maximum errors against the exact derivative ($v'(x) = \pi \cos(\pi x)e^{\sin(\pi x)}$);

N	Max error
4	$5.50 \cdot 10^{-1}$
8	$1.36 \cdot 10^{-2}$
16	$5.54 \cdot 10^{-7}$
32	$1.72 \cdot 10^{-13}$
64	$5.85 \cdot 10^{-13}$

We see that as N increases the error term drops exponentially, demonstrating spectral convergence for $v(x) \in C^\infty$.

e) We start by constructing the functions $w^1(x)$, $w^2(x)$, and $w^3(x)$. Since $w^i(x) = \frac{dw^{i+1}}{dx}$ we also have $w^{i+1}(x) = \int w^i(x)dx$, so we construct our functions by sequentially integrating the base function $w^0(x)$. For $w^1(x)$ we get

$$w^1(x) = \begin{cases} -\frac{\sin(\pi x)}{\pi} + c_1 & \text{for } -2 \leq x < 0 \\ \frac{\sin(\pi x)}{\pi} + c_2 & \text{for } 0 \leq x \leq 2 \end{cases}$$

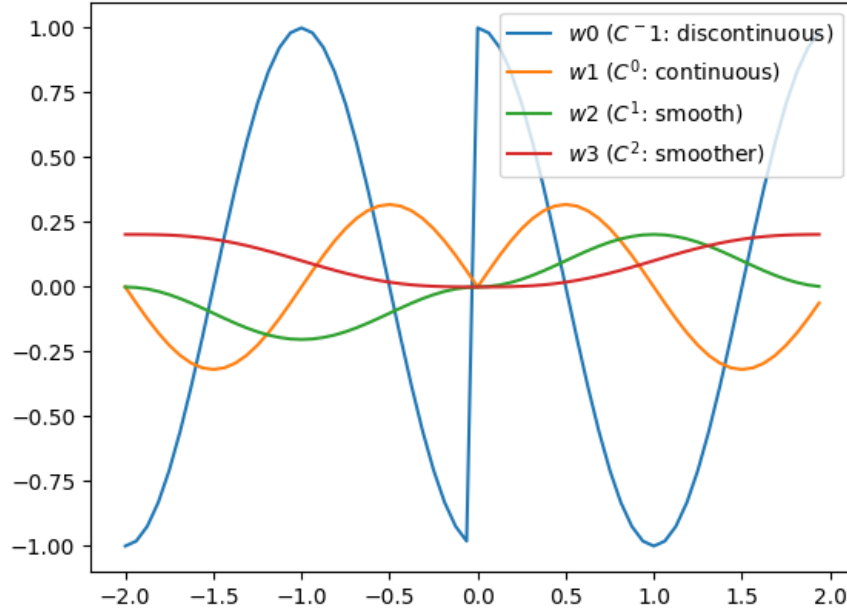
It is stated that $w^{i+1} \in C^i([-2, 2])$ such that $w^1(x) \in C^0([-2, 2])$, meaning the function should be a continuous function. To ensure this, we must make sure that both limits meet each other at $x = 0$. And as both limits are equal, $\frac{\sin(\pi \cdot 0)}{\pi} = -\frac{\sin(\pi \cdot 0)}{\pi} = 0$, we do not need to adjust c_1 and c_2 , we say $c_1 = c_2 = 0$. We continue this pattern for $w^2(x)$, and $w^3(x)$ and get

$$w^2(x) = \begin{cases} \frac{\cos(\pi x)}{\pi^2} - \frac{1}{\pi^2} & \text{for } -2 \leq x < 0 \\ -\frac{\cos(\pi x)}{\pi^2} + \frac{1}{\pi^2} & \text{for } 0 \leq x \leq 2 \end{cases}$$

$$w^3(x) = \begin{cases} \frac{\sin(\pi x)}{\pi^3} - \frac{x}{\pi^2} & \text{for } -2 \leq x < 0 \\ -\frac{\sin(\pi x)}{\pi^3} + \frac{x}{\pi^2} & \text{for } 0 \leq x \leq 2 \end{cases}$$

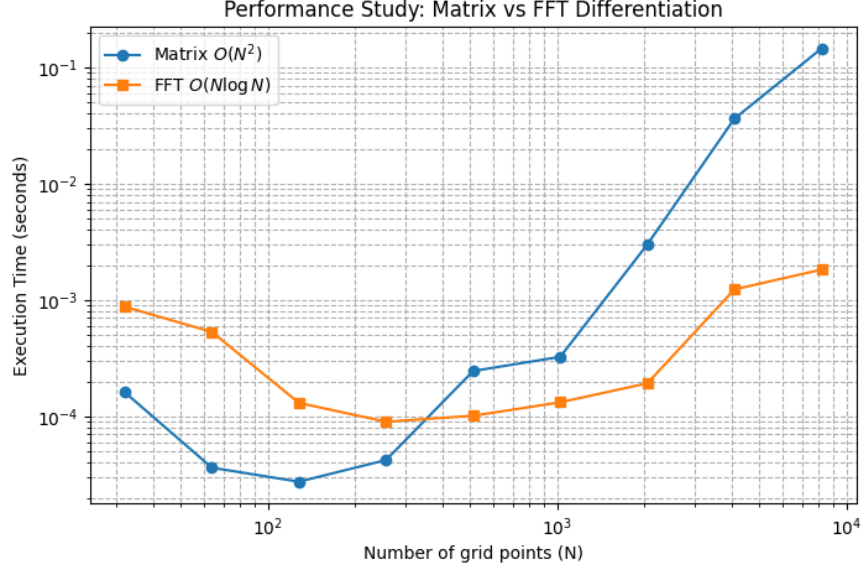
To compute the discrete derivative of these functions, we again need to scale our differentiation matrix $\mathcal{D}$. We use same methodology as in subpart d) to find a scaling factor of $\frac{\pi}{2}$, such that the discrete derivative is computed as $\frac{\pi}{2}\mathcal{D}w^i$. We measure the L^2 -norm of the error for each of these functions using the given definition of the L^2 -norm. But since this definition is continuous, and we want discrete L^2 -norms, we replace the integral with a Riemann sum rectangle approach such that the discrete L^2 -norm definition becomes $\sqrt{\sum u(x)^2 \Delta x}$, where Δx is the stepping size for N , and $u(x)$ is our error function. Since it is stated that $w^i(x) = \frac{dw^{i+1}}{dx}$, the theoretical derivative of the function $w^i(x)$ is therefore just the previous function $w^{i-1}(x)$. Therefore our error function is simply just the difference between the theoretical derivative and the numerical differentiation matrix product. Lastly, we expect the convergence rate of our functions will increase as i does, i.e. $w^3(x)$ has faster convergence than $w^2(x)$ &c, but we do not expect exponential spectral convergence as we have seen in the other exercises so far. Fourier spectral methods only achieve exponential convergence for infinitely smooth periodic functions (C^∞).

N	w^1 error	w^2 error	w^3 error
4	2.00	$4.50 \cdot 10^{-1}$	$6.15 \cdot 10^{-2}$
8	1.29	$1.37 \cdot 10^{-1}$	$1.13 \cdot 10^{-2}$
16	$8.97 \cdot 10^{-1}$	$4.73 \cdot 10^{-2}$	$2.09 \cdot 10^{-3}$
32	$6.32 \cdot 10^{-1}$	$1.66 \cdot 10^{-2}$	$3.76 \cdot 10^{-4}$
64	$4.46 \cdot 10^{-1}$	$5.87 \cdot 10^{-3}$	$6.68 \cdot 10^{-5}$



f) The fast Fourier transform represents a smooth function v as the finite sum of exponential waves on the form $e^{ik\theta}$ multiplied by their frequency amplitude $\hat{v}_k$. Since $\hat{v}_k$ is just some constant, the derivative of one of these exponential waves wrt θ is simply $ik \cdot e^{ik\theta}$, where k is still the frequency. We see that this is computationally straight forward compared to using a differentiation matrix, as taking the derivative of a function now reduces to multiplying the coefficients by ik for all k . So computing the derivative of $v(x) = e^{\sin(\pi x)}$ on the domain $x \in [0, 2]$ reduces to these steps;

1. We apply the `scipy.fft.fft` function to the data points $v(x_j)$ to get their coefficients $\hat{v}_k$.
2. We multiply each coefficient by ik . And because $v(x)$ is defined for the domain $x \in [0, 2]$, we must factor it by π to fit for the standard Fourier domain $\theta \in [0, 2\pi]$.
3. We then apply the inverse FFT in the function `scipy.fft.ifft` to the new coefficients such that we get back the derivative of the datapoints $v'(x_j)$.



While it depends on the seed, a breakpoint is usually observed between $N = 256$ and $N = 512$. Beyond this FFT begins to outperform the differentiation matrix. This is because the differentiation matrix operation, essentially multiplying a $N \times N$ matrix by a vector with N elements, has expected $O(N^2)$. But we know FFT achieves $O(N \log N)$. Thereby confirming that for $N \rightarrow \infty$, FFT outperforms the differentiation matrix multiplication by a huge margin, and for very large N , FFT is the only feasible of the two methods.

Polynomial Methods

h) We define a function to construct the Jacobi polynomial $P_n^{(\alpha, \beta)}(x)$ by sequentially constructing all the Jacobi polynomials from $P_0, \dots, P_n$. As Jacobi polynomials can be calculated from their neighbouring polynomials using the three-term recursion relation, we write our stepping function as

$$P_{n+1}(x) = \frac{(x - a_n)P_n(x) - b_{n-1}P_{n-1}(x)}{b_n}$$

Where a_n is the n 'th element in the diagonal of the Jacobi matrix, and b_n is the n 'th element in the super diagonal, we say $a_n = J_{(n,n)}$ and $b_n = J_{(n,n+1)}$. Both the Jacobi elements and the baseline element $P_0(x)$ is calculated according the same calculations as in the given MATLAB file `JacobiGQ.m`. For Chebyshev polynomials where $\alpha = \beta = -0.5$, we do however reach a division by zero error in the calculation of b_n , which we have to handle manually. We write out the entire term for our first calculation of b_0 ;

$$b_0 = \frac{2}{h_n + 2} \sqrt{\frac{1 \cdot (1 + \alpha + \beta)(1 + \alpha)(1 + \beta)}{(h_n + 1)(h_n + 3)}}$$

Since $h_0 = 2 \cdot 0 + \alpha + \beta = \alpha + \beta$;

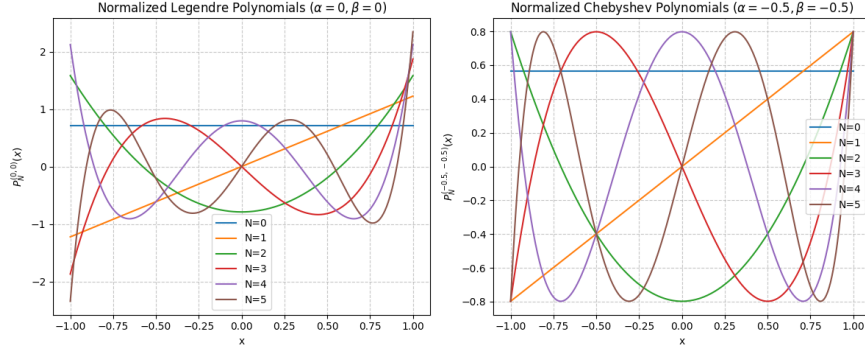
$$b_0 = \frac{2}{(\alpha + \beta) + 2} \sqrt{\frac{1 \cdot (1 + \alpha + \beta)(1 + \alpha)(1 + \beta)}{((\alpha + \beta) + 1)((\alpha + \beta) + 3)}}$$

When $\alpha = \beta = -0.5$ we have that $\alpha + \beta = -1$. Which gives us a zero term in both numerator and denominator in the second term. But we can resolve this by cancelling out the two terms, $\frac{a}{a} = 1$;

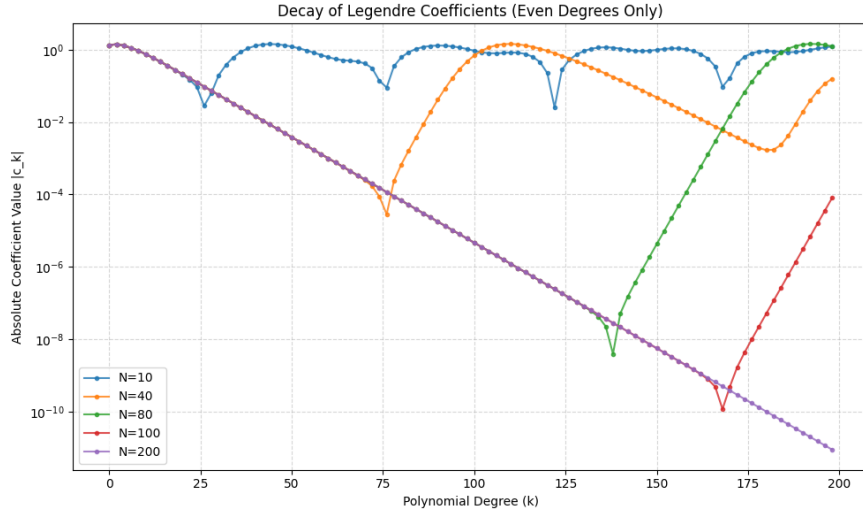
$$b_0 = 2\sqrt{\frac{(1+\alpha)(1+\beta)}{\alpha+\beta+3}}$$

$$b_0 = \sqrt{2(1+\alpha)(1+\beta)}$$

We then plot the first six polynomials to verify this implementation;



i) We compute the first 200 discrete Legendre coefficients using the discrete transform method across $N = 10, 40, 80, 100, 200$ nodes and visualize them on a logarithmic scale;

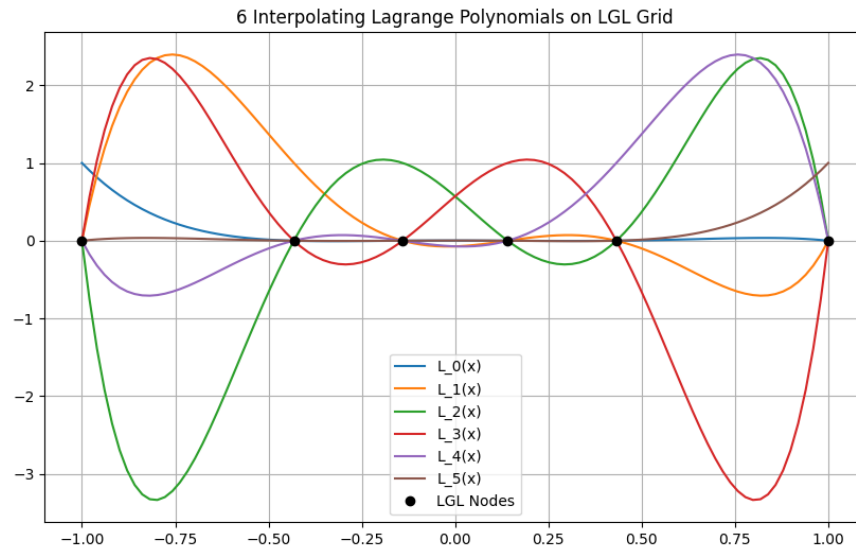


We observe that all odd-numbered coefficients are equal to 0, and therefore only plot even coefficients above. The reason this behaviour occurs is because $u(x)$ is symmetric around 0 while the odd-numbered polynomials are anti-symmetric. Their inner product over the symmetric domain $[-1, 1]$ is therefore zero. With these filtered out we observe that the coefficient values goes linearly down. Since the absolute coefficient value is plotted on a logarithmic scale this means they are dropping exponentially, and thereby showing exponential convergence. However, this accuracy is bounded by the number of nodes N . Gaussian quadrature theory states that this discrete transform method can only guarantee orthogonality if the sum of their degrees is $\leq 2N - 1$. Thus, the coefficient values start growing around this point for all values of N less than the 200 coefficients we computed in total.

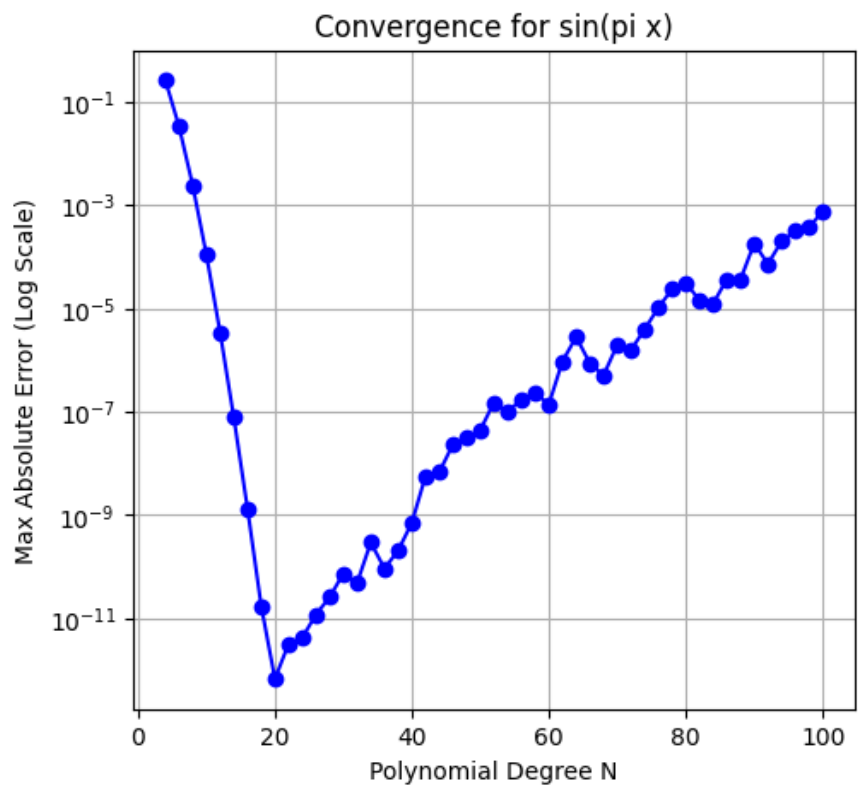
j) A Legendre Gauss Lobatto grid with 6 points includes the boundaries as well as the roots for the derivative of the Legendre polynomial. Since we want a 6 point grid, we need a 5th-degree Legendre polynomial, $P_5(x)$, to provide the 4 internal turnpoints. To generate these points we utilize the fact that the derivative of $P_5'(x)$ is a scalar of the Jacobi polynomial $P_4^{(1,1)}(x)$. The roots of $P_4^{(1,1)}(x)$ is therefore equal to the turnpoints of $P_5(x)$.

The definition of a Lagrange polynomial $L_k(x)$ is that it has height 1 at x_k and 0 at all other nodes. We can therefore define the 6 Lagrange polynomials evaluated at the 6 nodes as a 6×6 identity matrix, I . As the inverted Vandermonde matrix transformed the nodal heights of Legendre polynomials into their modal coefficients, we observe that $V^{-1}I = V^{-1}$. So we already have the Legendre coefficients required to construct each Lagrange polynomial in the columns of V^{-1} .

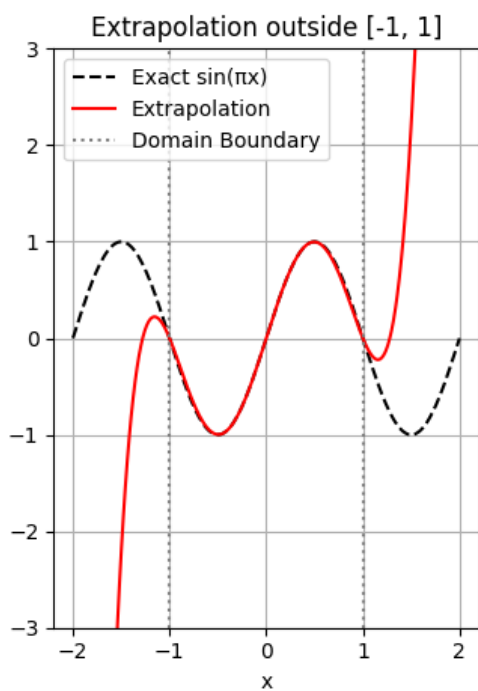
To visualize the interpolating Lagrange polynomials we construct a uniform grid containing 100 uniformly spaced points, $x_{\text{dense}} \in [-1, 1]$, and evaluate the Legendre polynomials at every point to get the 100×6 Vandermonde matrix, V_{dense} . Now we can compute $V_{\text{dense}}V^{-1}$ and get the Lagrange Vandermonde matrix, which we plot;



We project the Legendre coefficients of $v(x) = \sin(\pi x)$ onto x_{dense} using the $V_{\text{dense}}V^{-1}$ matrix, and evaluate the max absolute difference for increasing polynomial degrees. The error decreases linearly on a logarithmic scale, thereby showing spectral convergence. But around $N \approx 20$, the error starts to rise again. This is not due to some failure in the spectral method, but rounding errors during the matrix inversion;



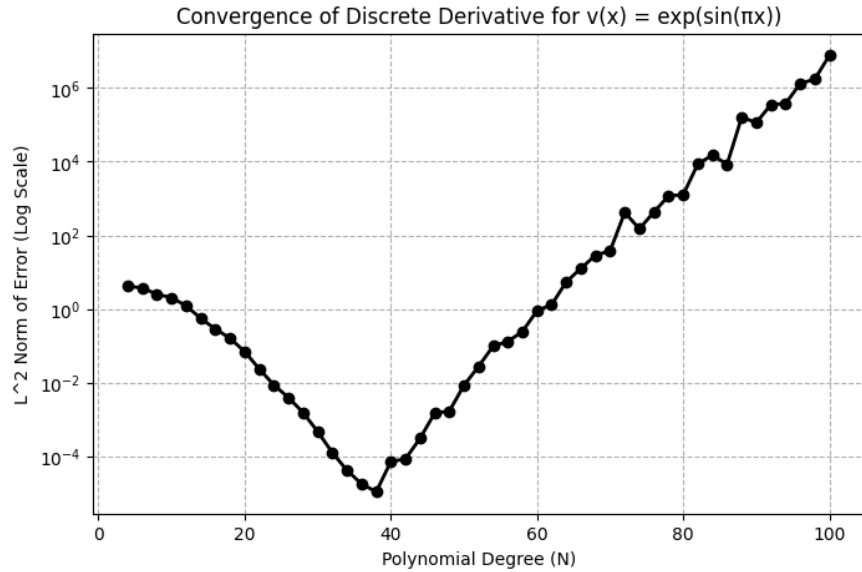
Lastly we project the coefficients from the 6 point grid onto an extended 200 point domain $x_{\text{extended}} \in [-2, 2]$;



We see that the polynomial captures behaviour of $v(x)$ perfectly inside the $[-1, 1]$ domain, but diverges off as soon as x crosses the boundaries. This is because the polynomial we construct is dominated by its highest order term. So as $|x|$ increases the polynomial grows rapidly. This means it can not extrapolate a bounded periodic function globally, and that the spectral interpolation we performed is only valid within the domain of the LGL nodes.

k) Since $v(x)$ from d) is defined on the domain $x \in [0, 2]$, we define the domain $r \in [-1, 1]$ as our reference domain for our Legendre polynomials. As we utilized in the previous exercise, the first derivative of a Jacobi polynomial $P_n^{(\alpha, \beta)}(r)$ is proportionally equal to $P_{n-1}^{(\alpha+1, \beta+1)}(r)$. So we construct the derivative Vandermonde matrix, V_x , from this and scale it by $\sqrt{j(j + \alpha + \beta + 1)}$ to account for the normalized Jacobi polynomials our function `jacobi_P()` generates. We can then define the differentiation matrix as $\mathcal{D} = V_x V^{-1}$, which takes physical function values at the nodes by V^{-1} , and applies the derivative to those coefficients by V_x . To evaluate the derivative of $v(x) = \exp(\sin(\pi x))$ on $x \in [0, 2]$ we define the mapping as $x = r + 1$, with the scaling factor $\frac{dr}{dx} = 1$. Thereby, we have that our differentiation matrix $\mathcal{D}$ already applies to the domain x without additional scaling.

For this, we measure the L^2 -norm of the derivative error across increasing N 'th degree polynomials. Because our Legendre basis is orthonormal, Parseval's identity states that the continuous spatial energy of the function is equal to the discrete sum of its squared modal coefficients. Thus, we can compute the continuous L^2 -norm simply as $\|c_e\|_2 = \sqrt{\sum c_e^2}$;



The discrete derivative exhibits exponential spectral convergence as we see the L^2 -error decreasing linearly on the logarithmic scale. But at $N \approx 38$, the error bottoms at almost 10^{-5} and diverges back up.

We reason this is partially driven by ill-conditioning, for which evaluating the condition number of the Vandermonde matrix V gives us that it is $\approx 10^7$ for $N = 38$.

l) We construct the mass matrix M on the reference domain $r \in [-1, 1]$. In this domain it evaluates the integral of the overlapping interpolating basis functions, and because our Legendre basis is orthonormal, this integration is just the product of its coefficients, $c^T c = I$. By substituting $c = V^{-1}u$ such that we get the modal coefficient, the integration in the physical space becomes $I = (V^{-1}u)^T (V^{-1}u) = u^T (VV^T)^{-1}u$. And since the integration with the mass matrix for a vector u follows the calculation, $u^T M u$, the mass matrix for the reference domain is thereby $M_{\text{ref}} = (VV^T)^{-1}$.

To perform the integration over some arbitrary interval $x \in [a, b]$ we need make a scalar mapping. The length of r is 2 and the length of x is $b - a$, so we multiply the ratio with these; $\frac{b-a}{2}$. Scaling the mass matrix for the reference domain by this yields the mass matrix for the physical domain, $M_{\text{phys}} = \frac{b-a}{2}(VV^T)^{-1}$.

We apply M_{phys} to compute the L^2 -norm as $\|u\|_{L^2} = \sqrt{u^T M_{\text{phys}} u}$ for $u_1(x) = 0.5$ and $u_2(x) = 2\sin(\pi x)$ on the interval $[0, 2]$ using $N = 6$ nodes. The computation for u_1 yields an exact solution, and for u_2 the solution is very close to the analytical. Matrix based integration like this is exact when we are integrating a polynomial of a degree no higher than $2N - 3$. This is because the matrix is equivalent to Gauss-Lobatto quadrature, built from N polynomial nodes. As $u_1^2 = 0.25$ is a polynomial with degree 0 within the limit, $2(6) - 3 = 9$, and therefore the solution for this is exact. But as u_2 is a sine wave with no finite polynomial degree, the squared function u_2^2 also has no finite polynomial degree, which makes our solution only an approximation.